Topics
More
CATEGORIES
Sensors
Data
General
Real-Time Map
Community Projects
All categories

https://community.purpleair.com/t/how-to-make-efficient-api-calls/6906

How to Make Efficient API Calls

Data API

PurpleAir 32 Nov 2023

Nov 2023
1 / 1
Nov 2023

> This article is related to the PurpleAir API. If you are unfamiliar with it, check out our API Landing Page to get started.

This article outlines many ways that you can reduce the points usage of your API calls. These can involve either lowering the base cost of the call, or avoiding querying unnecessary fields.

> Some of these tips or fields listed are only applicable to certain API endpoints

Nov 2023

AVERAGING PERIODS AND DATA RESOLUTION

The easiest way you can reduce your point cost is by querying a larger averaging period. For example, changing a request from real-time to hourly results in 30 times less data points, and a similar change in cost.

MODIFIED SINCE

When querying real-time data from Get Sensors Data 56 or Get Members Data 15 , the "modified_since" parameter can be used to only return data from sensors that have reported since a given timestamp. This can be used with the data_time_stamp for your last call to only receive newly reported data.

> The "modified_since" parameter is based on when a sensor has last reported data, while the "last_modified" field contains information on when a sensors's registration was last changed. Though similarly named, they operate differently.

For example, if we send a request to Get Sensors Data, the response should include an element of metadata called "data_time_stamp." If we set the modified_since parameter to the data_time_stamp of our last API call, then we will only receive data for sensors that have reported new data since the previous API call.

Note that data_time_stamp is different from the response's "time_stamp". time_stamp indicates the time when the response was sent, while data_time_stamp indicates when the response's data is current until. To accurately obtain all new data since the previous response, you will want to use data_time_stamp and query the API every minute.

AVERAGED FIELDS

Many fields, such as pm2.5_atm or temperature, have an averaged form of the field as well as a and b channels available. For historical data fields where A and B channels are available, we always recommend querying the A and B channels separately and averaging the data locally.

This is because if a sensor with two laser counters has an issue with one laser counter, such as a bug that has crawled inside or a failed fan, you will be able to verify data quality by comparing the two channels. This verification process is important for historical data because a sensor's Confidence Score 7 — `confidence` —is not available as historical metadata; it is a current data point only.

Additionally, the points cost may be lower since averaged fields cost 2 points per row, while individual channel data is 1 point per row. If a sensor has only reported "temperature_a", but no data for "temperature_b", then requesting the averaged field "temperature," will still deduct 2 points per row of data returned. However, requesting temperature_a and temperature_b separately will only deduct 1 point for temperature_a per row of data returned. Since there is no data for temperature_b, you will not be deducted any points for that field in the returned data.

> While many PurpleAir sensors report two channels 7 of PM data, only a few can report two channels of temperature, humidity, and pressure data. Currently, no PurpleAir sensors can report two channels of VOC data.

GROUP CALLS

If you are querying real-time data for many sensors at once, the group endpoints allow you to query data from many sensors at once. This requires that you first create a group 16 and add members to a group 6 . You can then obtain real-time data for all group members with a single request using the Get Members Data 15 endpoint.

This means that instead of the base cost being 1 point for every sensor you query, the base cost will instead only be 5 points for data from the entire group.

FIELDS THAT DON'T UPDATE OFTEN

The following fields don't change often, and typically don't require frequent API calls.

The field last_modified indicates when the registration for a sensor was last updated. You can keep track of this field for a sensor index, and only request registration-related information if the registration has been updated.

Field Name	When It Changes
date_created	This field will never change for a sensor index
position_rating	The position rating only changes when a device is connected to a new WiFi network or has its registration modified.
hardware	The hardware detected by the sensor doesn't change frequently, and typically only changes when a component fails
firmware_version	This will change when a device is updated to a newer firmware version. Devices will automatically update when newer firmware becomes available
last_modified	This indicates when the registration was last modified
name	This can be changed during registration
location_type	This can be changed during registration
private	This can be changed during registration
latitude	This can be changed during registration
longitude	This can be changed during registration
altitude	This can be changed during registration
led_brightness	This can be changed during registration

UNUSED OR RARELY USED FIELDS

Field Name	Description
ozone1	An extremely limited number of PurpleAir sensors have ever reported ozone data. At the time of writing, only five sensors can be seen on the map with the "Ozone (ppb)" data layer selected
icon	This is a flag reserved for future use to reference an icon for the sensor
pa_latency	This field measures the round trip time taken to send air quality data to PurpleAir and receive a response in milliseconds
memory	This value is primarily used by PurpleAir for diagnostic information of sensors
analog_input	This is the analog voltage on the ADC input of the PurpleAir sensor control board
confidence_manual	We recommend using the confidence field instead, as it takes confidence_manual into account
confidence_auto	We recommend using the confidence field instead, as it takes confidence_auto into account
primary_id_a	A legacy field from when PurpleAir used ThingSpeak
primary_key_a	A legacy field from when PurpleAir used ThingSpeak
secondary_id_a	A legacy field from when PurpleAir used ThingSpeak
secondary_key_a	A legacy field from when PurpleAir used ThingSpeak
primary_id_b	A legacy field from when PurpleAir used ThingSpeak
primary_key_b	A legacy field from when PurpleAir used ThingSpeak
secondary_id_b	A legacy field from when PurpleAir used ThingSpeak
secondary_key_b	A legacy field from when PurpleAir used ThingSpeak

CALCULATABLE DATA

The following fields can be derived from other sensor data using formulas available on the PurpleAir community forums. They are still available on the API, but API users concerned about cost can also locally calculate these values

Name	Source Equation
scattering_coefficient	Visibility Data Layers 19
deciviews	Visibility Data Layers 19
visual_range	Visibility Data Layers 19

LEARN MORE

API Landing Page
Data Download Tool 17
Making API Calls with the PurpleAir API 47

3

Purple Air API charges 19
Using the API Dashboard 4
Obtaining PurpleAir location data (lat/long) for California 3
Research Considerations 2
How to Get Historical Sensor Data Step-by-Step Using the Data Download Tool

1 MORE

2.1K VIEWS 13 LINKS

REPLY

Related topics

Topic	Replies	Activity
API Use Guidelines API	0	May 2024
API Usage - high points consumed for get_sensor endpoint Data	4	Jan 2024
How to Tell When A Sensor Produced Data API	0	May 2024
Q: How Do I Calculate My API Call's Point Cost FAQ	0	Jun 2024
API Pricing API	51	12d

© 2025, PurpleAir | Terms & Conditions - Privacy Policy